

Assignment 1

Priyanshi Dhillon^[2879065], Shithik Shaji^[2886944], and Vaishnavi Mehta^[2889259]

Vrije University, Amsterdam, NL

1 Introduction

Smartphone-based ecological momentary assessment offers a continuous, low-burden window into the mental health of individuals with depression. This report analyses data from 27 participants over 31–69 days. A smartphone application recorded passive behavioural signals (screen time, app usage, activity, communication) alongside self-reported mood, arousal, and valence. The goal is to predict next-day average mood per participant.

The analysis follows the CRISP-DM process model[1]. Section 2 covers exploratory data analysis, cleaning, and feature engineering. Section 3 presents two classification approaches for discretised mood prediction, one instance-based and one temporal. Section 4 describes an association rule extension to the Apriori algorithm. Section 5 applies regression to the continuous mood target. Section 6 examines the properties and practical impact of the chosen evaluation metrics.

2 Data Preparation

2.1 Exploratory Data Analysis

The dataset comes from a mental health smartphone study involving 27 participants (IDs AS14.01–AS14.33), who provided both self-reported information and passive sensor data over periods ranging from 31 to 69 days. After transforming the raw data from a long format into a daily wide format, and keeping only the entries where mood was recorded, the final dataset contains 1,268 records with 19 feature columns.

Table 1: Dataset overview

Property	Value
Total records (with mood)	1,268
Unique participants	27
Number of features	19
Data collection span per participant	31–69 days
Mood scale	1–10
Circumplex arousal / valence scale	–2 to 2
Activity scale	0–1

The 19 features divide into four types: two binary communication indicators (`call`, `sms`), twelve application-usage duration columns (`appCat.*`), four continuous behavioural/psychological measures (`activity`, `screen`, `circumplex.arousal`, `circumplex.valence`), and the prediction target `mood`.

Missing values. Missingness varies dramatically across columns (Table 2). Core psychological measures (`circumplex.arousal`, `mood`) are fully observed, while infrequently used app categories such as `appCat.weather` (91.2% missing), `appCat.game` (85.1%), and `appCat.unknown` (79.9%) are absent for the vast majority of days (Fig. 1). This pattern is not random: a zero-usage day leaves no record, so absence directly encodes non-use rather than a data-collection failure.

Table 2: Missing value rates for selected features

Feature	Missing (%)	Feature	Missing (%)
appCat.weather	91.2	appCat.social	24.1
appCat.game	85.1	appCat.entertainment	35.1
appCat.unknown	79.9	screen	7.9
appCat.office	78.7	activity	9.3
appCat.travel	67.0	circumplex.valence	0.2
appCat.utilities	66.6	circumplex.arousal	0.0
sms	66.2	mood	0.0

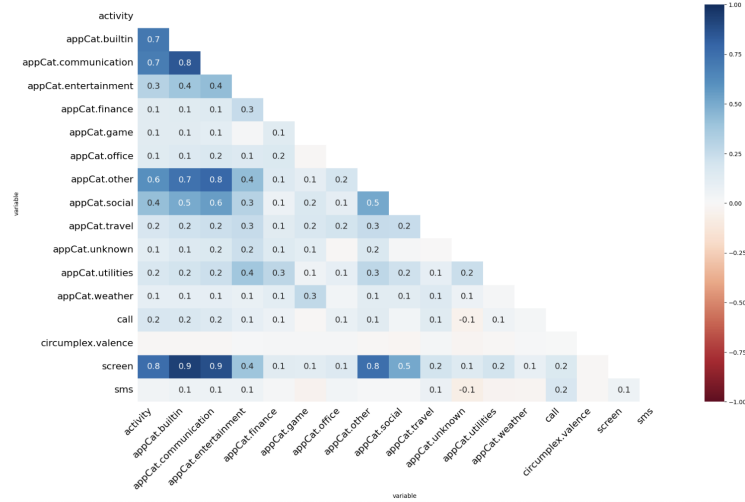


Fig. 1: Missing value matrix.

Outliers. An IQR-based scan reveals substantial numbers of extreme values in the `appCat.*` columns, with `appCat.other` (124 flagged rows), `appCat.builtin` (110), and `activity` (91) showing the most. One record in `appCat.builtin` contains a physically impossible negative value (-772.8 seconds), confirming the presence of data-entry errors.

Temporal irregularities. Mood observations vary per participant. Gap analysis shows one major 22-day gap (AS14.01), a 12-day gap (AS14.12), and otherwise gaps 7 days.

Relationships. A Pearson correlation heatmap computed on the 19 numeric features identifies `circumplex.valence` as the variable most strongly correlated with `mood`. A scatter plot with a linear fit (Fig. 2) confirms a clear positive linear relationship between the two, consistent with the circumplex model of affect. Per-patient mean mood ranges from approximately 6 to 8 (Fig. 3), reflecting both genuine individual differences and variation in data-collection duration.

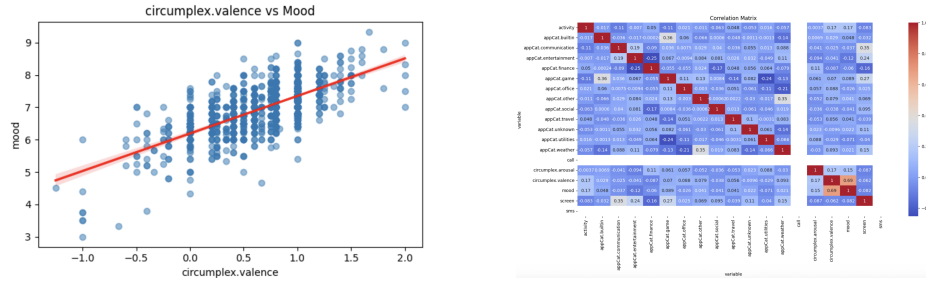


Fig. 2: (Left) Valence vs mood scatter plot. (Right) Correlation heatmap.

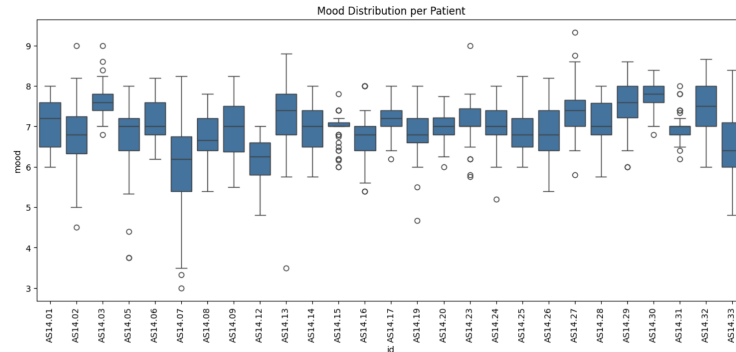


Fig. 3: Boxplot of mood per participant.

Screen vs. app usage discrepancy. The sum of all `appCat.*` durations is systematically larger than the `screen` column (maximum app-sum: 3,809.9 s vs. maximum screen: 960.5 s). `screen` captures foreground usage, while app categories may include background activity, so both metrics are not directly comparable.

2.2 Data Cleaning

Incorrect Value Removal Domain constraints define hard boundaries for each variable. A constraint check across all 1,268 rows found no violations in the psychological scores. However, one row contained a negative value for `appCat.builtin` (−772.8 s), which was replaced with NaN rather than removed, preserving the remaining valid information in that row. No duplicate records were detected.

Extreme Value Treatment Beyond incorrect values, the highly right-skewed distributions of app-usage columns necessitate further treatment. Two complementary strategies were applied.

For the `screen` column, per-user Winsorization at the 95th percentile was used [3, 4]. This approach caps each user’s extreme screen-time values at their own 95th-percentile threshold rather than a global one, respecting individual variation.

For all twelve `appCat.*` columns, a $\log(1 + x)$ transformation was applied [2, 9]. These columns exhibit heavy right tails; the $\log_{1p}$ transform compresses high-magnitude values while preserving zero entries and monotonic relationships.

Missing Value Imputation Since the dataset has a time-series structure, two imputation strategies suited for temporal data were explored [5].

Before applying imputation, rows with more than 13 missing values (out of 19 features) were removed, reducing the dataset to 1,174 rows. In addition, long gaps of more than 4 consecutive days were treated as breaks in the data, and only continuous segments with at least 10 consecutive days were kept [7, 5]. Short gaps of 1–4 days were considered suitable for imputation.

Method 1 (selected) uses semantics-aware imputation tailored to each variable. `AppCat.*` columns are filled with zero (no usage), call/sms with per-participant mode, activity/screen with participant averages, and psychological variables by averaging forward- and backward-fill. **Method 2 (alternative)** applies uniform forward-fill followed by backward-fill across all columns.

Method 1 was chosen because it better reflects how different data types are generated, yielding more realistic imputations [7, 8, 5]. Method 2 risks carrying app usage into days where no usage occurred. After applying Method 1, the final cleaned dataset consists of 1,214 rows with no missing values.

2.3 Feature Engineering

The cleaned dataset represents daily sensor snapshots. To enable a standard supervised learning approach, the history for each participant was aggregated into a fixed-length feature vector per prediction target [8, 6].

Interaction features. Two interaction terms were created: `valence_x_arousal` and `activity_x_screen`, capturing joint affective and behavioural relationships.

Aggregate usage feature. Total app usage (`total_app_usage`) was computed as the sum of all log1p-transformed `appCat.*` columns. A social-usage ratio (`social_ratio`) was also derived.

Within-participant standardisation. `mood`, `screen`, and `total_app_usage` were z-scored per participant.

Windowed instance creation. A sliding window of width 3 days [8] was used to construct one training instance per day per participant. Each instance includes lag, rolling, and trend features. The target is same-day mood. After filtering, the dataset contains 1,133 instances and 24 features.

3 Classification

3.1 Target Definition and Evaluation Setup

Next-day mood (range ≈ 3 –9.3) is discretized into three classes (Low, Medium, High). Two binning strategies were explored. The primary approach uses data-driven percentile thresholds (33% and 66%) computed on the training set only to prevent data leakage resulting in three balanced classes. A fixed binning strategy (0–5.5, 5.5–7.5, 7.5–10) was also tested and had some meaningful partition, but we maintain the first approach as the primary focus. Therefore, percentile-based binning is used for all reported results.

Each participant’s records were sorted chronologically and split such that the final 20% of observations form the test set, with the remaining 80% used for training. This split is performed per participant and then pooled.

The primary evaluation metric is weighted F1, which accounts for class imbalance by weighting each class by its support. Macro F1 is also reported to evaluate performance uniformly across classes, particularly to detect failures on minority classes.

3.2 Non-Temporal Classification

Model Description. XGBoost is a gradient-boosted tree ensemble where models are trained sequentially, with each new tree correcting the residual errors of the previous ensemble. This allows it to capture non-linear relationships and interactions effectively while controlling overfitting through regularization [14].

Random Forest was used as a baseline (top-10 features), but XGBoost performed slightly better and was selected



Fig. 4: The resulting three-class distribution in the training split after quartile-based binning

Hyperparameter Optimisation. A grid search was conducted over the following parameter space: `n_estimators` $\in \{100, 200, 300\}$, `max_depth` $\in \{3, 5, 7\}$, `learning_rate` $\in \{0.01, 0.1, 0.2\}$, and `subsample` $\in \{0.6, 0.8, 1.0\}$.

The optimal configuration was: `learning_rate` = 0.01, `max_depth` = 3, `n_estimators` = 300, `subsample` = 1.0.

Evaluation Setup This non-temporal model operates on the fixed-length feature vectors constructed in Task 1C, thereby satisfying the requirement of applying a standard machine learning algorithm to the engineered dataset.

A random train-test split is inappropriate here because it would allow future observations to appear in training, violating the temporal ordering of the prediction task so the same temporal train-test split described previously is used. Models are trained on the training partition and evaluated on the held-out test set. The primary metric is weighted F1, with macro F1 and accuracy reported for completeness.

Table 3: Classification results on test set

Overall model performance				Per-class performance (XGBoost)			
Model	W-F1	M-F1	Acc.	Class	Precision	Recall	F1
Random Forest (top 10 features)	0.495	0.485	0.502	Low	0.56	0.70	0.62
				Mid	0.41	0.30	0.34
				High	0.63	0.61	0.62
XGBoost(all-feat) (all 24 features)	0.537	0.527	0.550	Macro avg	0.53	0.53	0.53

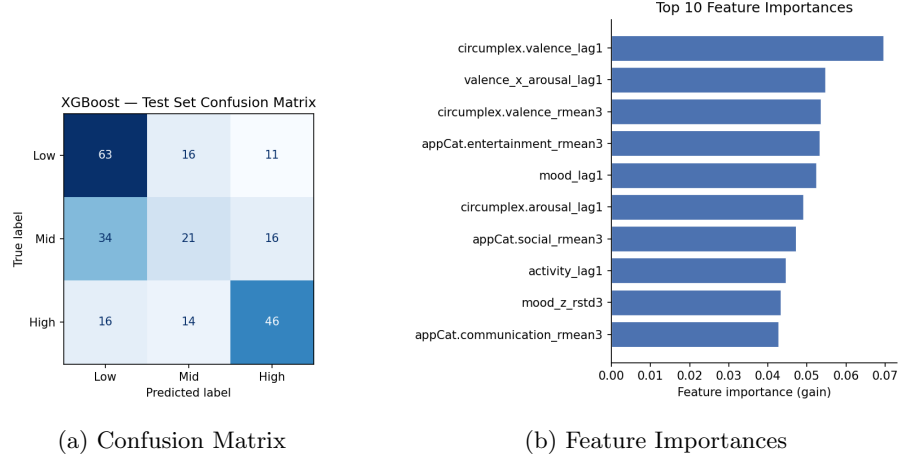


Fig. 5: XGBoost classifier performance: (a) test-set confusion matrix and (b) top 10 feature importances.

Results. The medium-mood class (class 1) is consistently the most difficult, with an F1 of 0.34. This reflects that the percentile thresholds place the middle class within the narrowest numerical range (6.80–7.25), concentrating instances close to both boundaries and making separation from adjacent classes inherently ambiguous. Low and high mood are predicted more reliably ($F1 \approx 0.62$ for both).

Misclassifications in the confusion matrix (Fig. 5a) are predominantly between adjacent classes, consistent with the ordinal and continuous nature of the underlying variable. Figure 5b shows that the model relies most heavily on recent mood-related and affective features, particularly lagged mood and circumplex variables. This indicates that short-term emotional state is the strongest predictor of next-day mood, while app usage features play a secondary role.

Interpretation The overall performance level is consistent with prior work on smartphone-based mood prediction; Asselbergs et al. [8] reports comparable accuracy on analogous datasets, reflecting that next-day mood from passive sensing alone is genuinely difficult to predict. XGBoost’s advantage over the Random Forest baseline suggests that sequential error correction captures non-linearities in the feature–mood relationship that bagging alone does not, though the gain is modest, indicating the relationship is not strongly non-linear at this dataset scale.

3.3 Temporal Classification

Model Description The model used is a pure LSTM Classifier. The LSTM receives a sliding window of five consecutive daily observations[10] for each participant as its input sequence. All input features are standardized using a

`StandardScaler`[11] fitted exclusively on the training partition and then applied to the validation and test sets. The final hidden state of the last LSTM layer is passed through a `LayerNorm` step [13] and into a two-layer feed-forward head that produces logit scores over the three classes, trained with standard `CrossEntropyLoss` [12]

Hyperparameter Optimisation Hyperparameter selection is performed on a grid over four parameters: number of hidden units per LSTM layer ($\{16, 32, 64\}$), number of stacked LSTM layers ($\{1, 2\}$), dropout rate ($\{0.1, 0.2, 0.3\}$), and learning rate ($\{10^{-3}, 5 \times 10^{-4}\}$). Each candidate configuration is trained for up to 20 epochs per fold with an early-stopping patience of 5 epochs, and the optimisation objective is macro-averaged F1 score.

The best configuration identified was `lstm_hidden=64, lstm_layers=2, dropout=0.2, lr=1e-3`. This was then used to retrain the model from scratch on the full training split for up to 50 epochs, this time with a patience of 10 epochs monitoring validation loss.

Evaluation Setup Data are split per participant and chronologically. The first 80% of each participant’s timeline as training set, the next 10% the validation set, and the final 10% the test set. Importantly, sliding windows are constructed before splitting. Evaluation metrics are **accuracy** and **macro-averaged F1 score**. Accuracy provides an overall correctness measure, while macro-F1 is the primary criterion because it explicitly accounts for class by averaging per-class F1 scores with equal weight.

Table 4: LSTM Classifier performance across data splits.

Split	Accuracy	Macro F1
Train	0.6026	0.5861
Validation	0.5596	0.5456
Test	0.517	0.4917

Results The final model achieves a test accuracy of approximately 51% and a macro-F1 of around 0.49 (Table 4). The training curve in Figure 6a shows that there is a gap between the training and validation loss, with early stopping triggering around epoch 32. The gap between training and test performance shows that the model isn’t generalizing very well, suggesting some overfitting.

The confusion matrix (Figure 6b) reveals that the model handles the *Low* and *High* classes somewhat better than *Mid*. Per-class precision, recall, and F1 on the test set are presented in Table 5.

Interpretation and Rationale The LSTM model achieves average classification performance overall. Inspecting the confusion matrix reveals that Low and High

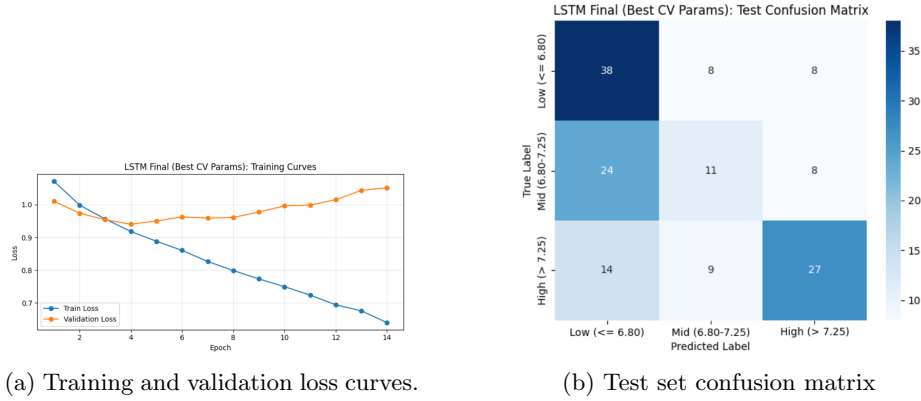


Fig. 6: LSTM Performance.

Table 5: Per-class classification report on the test set.

Class	Precision	Recall	F1
Low	0.50	0.70	0.58
Mid	0.39	0.26	0.31
High	0.63	0.54	0.58
Macro avg	0.50	0.51	0.49

mood classes are predicted reliably; the primary difficulty lies with the boundary class, whose narrow range (6.8–7.25) makes it inherently ambiguous. This is compounded by inter-individual variation in mood baselines, which limits the model’s ability to generalise across subjects. The training-to-test accuracy gap of roughly 9 percentage points suggests the model has captured genuine patterns in the data rather than overfitting.

3.4 Winning Classification Algorithm

The BirdCLEF 2023 competition, hosted on Kaggle in collaboration with the Cornell Lab of Ornithology, focused on identifying bird species from short audio clips [18]. Participants were given 5 second segments from longer soundscape recordings and had to predict which of the 264 Kenyan bird species were present. Since multiple species could occur at once, this was a multi-label classification task.

The training data included around 16,900 recordings from Xeno-canto, while the test set consisted of long soundscapes recorded in Baringo County, Kenya [19]. Performance was evaluated using padded class mean Average Precision (pcmAP), a modified version of cmAP designed to handle class imbalance by adding small artificial positives for each class. Models also had to meet a two-hour CPU inference limit.

The winning solution by Volodymyr Sydorskyi used an EfficientNet based model applied to mel spectrograms, combined with a Sound Event Detection (SED) framework [20]. While this modelling approach was common, the key difference was his focus on data quality. He manually reviewed recordings for rare species, removing noisy or incorrect samples and upsampling underrepresented classes.

He also used pseudo labelling, where an initial model generated predictions for unlabelled soundscapes, which were then added to the training data for further refinement. This helped the model adapt to real world conditions.

Overall, the success of this approach came less from model innovation and more from careful data cleaning and handling of distribution differences. It highlights that improving data quality can be more impactful than complex model tuning in real-world machine learning tasks.

Compressed to fit 1 page while keeping correctness and LNCS compliance:
“`latex`”

4 Association Rules

Association rule mining is used to identify interpretable relationships between behavioural features and mood states, focusing on frequent co-occurrence rather than predictive accuracy.

4.1 Methodology

A taxonomy-based Apriori algorithm [15,21] is applied. Continuous variables (mood, valence, activity, screen time) are discretised into Low/Medium/High using percentile thresholds consistent with prior tasks. App usage is grouped by behavioural function (e.g. Social, Entertainment, Communication) to reduce sparsity.

Each day is treated as a transaction of categorical items. Rules are generated with minimum support 0.05, confidence 0.6, and lift > 1 , and evaluated using these metrics.

4.2 Results

Table 6 shows the top rules by lift.

Table 6: Top association rules.

Rule	Supp.	Conf.	Lift
Low Mood $\wedge$ Low Valence $\Rightarrow$ Low Communication	0.052	0.602	1.860
Low Mood $\wedge$ Low Entertainment $\Rightarrow$ Low Communication	0.052	0.602	1.860
High Communication $\wedge$ High Entertainment $\Rightarrow$ High Mood	0.056	0.604	1.849
Low Entertainment $\wedge$ Low Communication $\Rightarrow$ Low Mood	0.052	0.619	1.630
Low Social $\wedge$ Low Screen Time $\Rightarrow$ Low Mood	0.056	0.604	1.591

The rules show consistent patterns: (i) low mood co-occurs with reduced communication and engagement, (ii) high communication and entertainment usage co-occur with high mood, and (iii) broadly low activity (communication, entertainment, social, screen time) is associated with low mood.

4.3 Advantages and Limitations

The primary advantage of association rules is interpretability. Unlike black-box models such as LSTMs, rules provide direct insight into behavioural patterns that may inform interventions.

However, there are several limitations:

- Discretisation introduces information loss
- Rules capture correlation rather than causation
- Sparse or infrequent behaviours may yield unstable rules

Overall, association rules complement predictive models by providing transparent, human-interpretable structure in the data, but should not be used in isolation for inference or prediction.

5 Numerical Prediction

5.1 Non-Temporal Regression

Model Description Ridge Regression minimises the residual sum of squares subject to an L2 penalty on the coefficient magnitudes. Lagged mood and behavioural features justify a linear model; regularisation prevents overfitting. Random Forest Regression was also evaluated as a non-parametric alternative that captures non-linear interactions without explicit specification; Ridge outperformed it on both MSE and MAE and was selected as the primary model.

Hyperparameter Optimisation

Hyperparameter Optimisation. The regularisation strength α was tuned over $\{0.1, 1.0, 10.0\}$ using five-fold cross-validation with negative MSE as the scoring metric. The optimal value was $\alpha = 10.0$.

For comparison, Random Forest was tuned over: `n_estimators` $\in \{100, 200\}$, `max_depth` $\in \{5, \text{None}\}$, `min_samples_split` $\in \{2, 5\}$.

Results. Table 7 shows that Ridge Regression achieves the lowest error, with $\text{MSE} = 0.401$ and $\text{MAE} = 0.485$, outperforming Random Forest ($\text{MSE} = 0.456$, $\text{MAE} = 0.512$) and the naive baseline.

Figure 7 shows that predictions follow a diagonal trend but are compressed toward the mean, underestimating extremes. The residual plot (Fig. 7b) shows that errors are smallest in the mid-range of mood values and increase at the extremes, with the model underestimating high mood values and overestimating low ones, a regression-to-the-mean effect also observed in the LSTM regressor [17].

Table 7: Non-temporal regression performance on the test set.

Model	MSE	MAE	RMSE
Naive baseline (train mean)	0.525	0.546	0.725
Random Forest Regressor	0.456	0.512	0.676
Ridge Regression	0.401	0.485	0.634

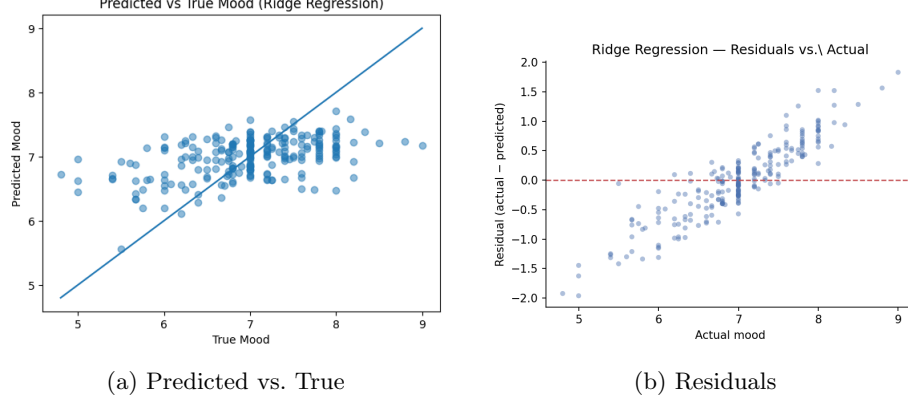


Fig. 7: Ridge Regression evaluation on the test set: (a) predicted vs. true mood values and (b) residuals vs. actual mood.

Interpretation The superior performance of Ridge suggests that the relationship between the engineered lag-based features and next-day mood is largely linear. The feature engineering step already captures temporal dynamics through aggregation, reducing the advantage of non-linear models.

Residual analysis shows a regression-to-the-mean effect[17], where extreme mood values are underpredicted or overpredicted toward the centre. This indicates that while overall error is low, the model is less reliable at the extremes of the mood range.

5.2 Temporal Regression

Main Approach The architecture is an **LSTM Regressor** structurally identical to the classifier but with a scalar output head (a single linear unit with no activation) trained under MSE loss instead of cross-entropy. Data partitioning follows Task 2, with MSE as the primary evaluation metric across train, validation, and test sets. For comparison with classification, regression outputs are binned using the same thresholds to compute accuracy and macro-F1, isolating the effect of optimisation approach. Hyperparameter search for the regressor uses a CV grid search over the validation set directly, optimizing over hidden size ($\{16, 32, 64, 128\}$), number of layers ($\{1, 2\}$), dropout ($\{0.1, 0.2, 0.3\}$), and learning rate ($\{10^{-3}, 5 \times 10^{-4}\}$). Each configuration is trained for up to 35 epochs with patience 8, and ranking is by validation MSE.

Split	MSE
Train	0.4369
Validation	0.4469
Test	0.4930

Table 8: LSTM Regressor performance.

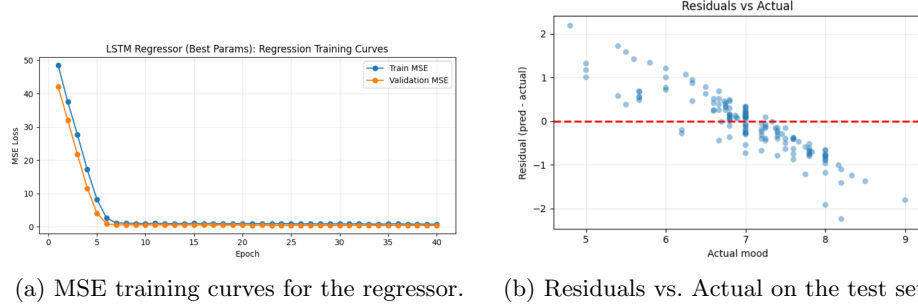


Fig. 8: LSTM Regressor training dynamics and test-set predictions

Results The training curve (Figure 8) has a steep initial descent followed by a plateau, with validation loss stabilising a few epochs after training loss. The Residuals vs Actual scatter (Figure 8, right) reveals a recognisable pattern that predictions are compressed toward the centre of the mood range, underestimating high values and overestimating low ones.

Interpretation. Comparing the two regression approaches directly: Ridge (MSE = 0.401) outperforms the LSTM (MSE = 0.493) on the test set, despite the LSTM operating on the richer sequential data, mirroring the pattern seen in classification, where the instance based model matched the temporal one since the feature engineering step in Task 1C already captures the temporal signal. Regression retains more information than classification by avoiding discretizations and the continuous target is estimated with reasonable precision. The trade-off between regression and classification is between label granularity and loss behavior. While regression leverages continuous labels, MSE bias toward the mean can reduce classification accuracy. It is still preferable when small differences (3.45 vs 3.60) carry meaningful signal lost in class binning.

6 Evaluation

6.1 Characteristics of Evaluation Metrics

For n samples with true values y_i and predictions $\hat{y}_i$:

$$\text{MSE} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2, \quad \text{MAE} = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|$$

Metric	Train	Test
Accuracy	0.4459	0.3673
Macro-F1	0.4412	0.3371

Table 9: Performance metrics for binned regression

MSE penalises large errors strongly and is sensitive to outliers, while MAE is linear and more robust. The two metrics coincide only when every residual has magnitude exactly 0 or 1, since $|e_i| = e_i^2$ holds solely at those points.

6.2 Impact of Evaluation Metrics

Table 10 reports MSE and MAE for the non-temporal models. Ridge is preferred under both metrics, so the choice of metric does not affect model selection. The RMSE/MAE ratio is 1.306 for Ridge and 1.321 for Random Forest, indicating Random Forest produces more large-magnitude errors.

Table 10: Regression performance comparison

Non-temporal models			Temporal models		
Model	MSE	MAE	Set	MSE	MAE
Random Forest Regressor	0.456	0.512	Train	0.43	0.4820
Ridge Regression	0.401	0.485	Validation	0.43	0.4931
			Test	0.49	0.5430

Ridge MSE is notably higher at extreme mood values (0.552–0.540) than at mid-range values (0.063), reflecting regression toward the mean at the tails, precisely where clinical intervention decisions are made. MSE surfaces this failure more clearly than MAE alone would.

For the deep learning model, test MSE and MAE are 0.49 and 0.543 respectively, with RMSE ≈ 0.700 . The narrow RMSE–MAE gap confirms no extreme outliers; errors cluster consistently near half a mood-scale point. Since MSE $<$ MAE across all splits, most residuals fall below 1.0 unit, consistent with $x^2 < |x|$ for $|x| < 1$. A typical daily prediction error of ~ 0.54 points reflects stable baseline tracking, and optimizing via MSE ensures the model penalizes severe mood-shift misses — critical in a healthcare context. An example situation is a binary prediction task where a model consistently predicts 0 for a dataset where the ground truth is always 1.

References

1. Wirth, R., Hipp, J.: CRISP-DM: Towards a standard process model for data mining. In: Proceedings of the 4th International Conference on the Practical Application of Knowledge Discovery and Data Mining, vol. 1, pp. 29–39 (2000)
2. Moshe, I., Terhorst, Y., Asare, K.O., Sander, L.B., Ferreira, D., Baumeister, H., Mohr, D.C., Pulkki-Råback, L.: Predicting symptoms of depression and anxiety using smartphone and wearable data. *Frontiers in Psychiatry* **12**, 625247 (2021)
3. Sharma, S., Chatterjee, S.: Winsorization for robust Bayesian neural networks. *Entropy* **23**(11), 1546 (2021)
4. Ghosh, D., Vogt, A.: Outliers: An evaluation of methodologies. *Joint Statistical Meetings Proceedings*, pp. 3455–3460 (2012)
5. Toye, A.A., Celik, A., Kleinberg, S.: Benchmarking missing data imputation methods for time series using real-world test cases. In: Proceedings of the 6th Conference on Health, Inference, and Learning (CHIL). PMLR **287**, pp. 480–501 (2025)
6. Jafarlou, S., Lai, J., Azimi, I., Mousavi, Z., Labbaf, S., Jain, R.C., Dutt, N., Borelli, J.L., Rahmani, A.: Objective prediction of next-day’s affect using multimodal physiological and behavioral data: Algorithm development and validation study. *JMIR Formative Research* **7**, e39425 (2023)
7. Jaques, N., Taylor, S., Sano, A., Picard, R.: Multimodal autoencoder: A deep learning approach to filling in missing sensor data and enabling better mood prediction. In: 7th International Conference on Affective Computing and Intelligent Interaction (ACII), pp. 202–208 (2017)
8. Asselbergs, J., Ruwaard, J., Ejdy, M., Schrader, N., Sijbrandij, M., Riper, H.: Mobile phone-based unobtrusive ecological momentary assessment of day-to-day mood: An explorative study. *Journal of Medical Internet Research* **18**(3), e72 (2016)
9. Zheng, A., Casari, A.: Feature Engineering for Machine Learning. O’Reilly Media (2018)
10. Canzian, L., Musolesi, M.: Trajectories of depression: Unobtrusive monitoring of depressive states by means of smartphone mobility traces analysis. In: Proceedings of the ACM International Joint Conference on Pervasive and Ubiquitous Computing (UbiComp), pp. 1293–1304 (2015)
11. Géron, A.: Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow, 2nd edn. O’Reilly Media, Sebastopol (2019)
12. Goodfellow, I., Bengio, Y., Courville, A.: Deep Learning. MIT Press, Cambridge (2016)
13. Ba, J.L., Kiros, J.R., Hinton, G.E.: Layer normalization. arXiv:1607.06450 (2016)
14. Chen, T., Guestrin, C.: XGBoost: A scalable tree boosting system. In: Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD), pp. 785–794 (2016)
15. Agrawal, R., Srikant, R.: Fast algorithms for mining association rules. In: Proceedings of the 20th International Conference on Very Large Data Bases (VLDB), pp. 487–499 (1994)
16. Dougherty, J., Kohavi, R., Sahami, M.: Supervised and unsupervised discretization of continuous features. In: Proceedings of the 12th International Conference on Machine Learning (ICML), pp. 194–202 (1995)
17. Barnett, A.G., van der Pols, J.C., Dobson, A.J.: Regression to the mean: What it is and how to deal with it. *Int. J. Epidemiol.* **34**(1), 215–220 (2005)
18. Klinck, H., Demkin, M., Dane, S., Kahl, S., Denton, T., Ramesh, V.: BirdCLEF 2023 Kaggle Competition. Available at: <https://www.kaggle.com/competitions/birdclef-2023> (2023)

19. Kahl, S., Denton, T., Klinck, H., et al.: Overview of BirdCLEF 2023: Automated Bird Species Identification in Long-form Soundscapes. CLEF 2023 Working Notes, CEUR Workshop Proceedings, vol. 3497 (2023)
20. Sydorskyi, V.: 1st Place Solution: Correct Data is All You Need. Kaggle BirdCLEF 2023 Write-up (2023), Available at: <https://www.kaggle.com/competitions/birdclef-2023/writeups/volodymyr-1st-place-solution-correct-data-is-all-you-need>
21. Han, J., Fu, Y.: Discovery of multiple-level association rules from large databases. In: VLDB (1995)